МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4
по дисциплине «Объектно-Ориентированное Программирование»
Тема: Шаблонные классы

Студент гр. 3341 _____ Максимова Е.Д.
Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2024

Цель работы.

Создать шаблонный класс управления игрой, шаблонный класс отображения игры, класс, считывающий ввод пользователя и класс, отвечающий за отрисовку поля.

Задание.

Создать шаблонный класс управления игрой. Данный класс должен содержать ссылку на игру. В качестве параметра шаблона должен указываться класс, который определяет способ ввода команда, и переводящий введенную информацию в команду. Класс управления игрой, должен получать команду для выполнения, и вызывать соответствующий метод класса игры.

Создать шаблонный класс отображения игры. Данный класс реагирует на изменения в игре, и производит отрисовку игры. То, как происходит отрисовка игры определяется классом переданном в качестве параметра шаблона.

Реализовать класс считывающий ввод пользователя из терминала и преобразующий ввод в команду. Соответствие команды введенному символу должно задаваться из файла. Если невозможно считать из файла, то управление задается по умолчанию.

Реализовать класс, отвечающий за отрисовку поля.

Выполнение работы.

UML Диаграммы классов:

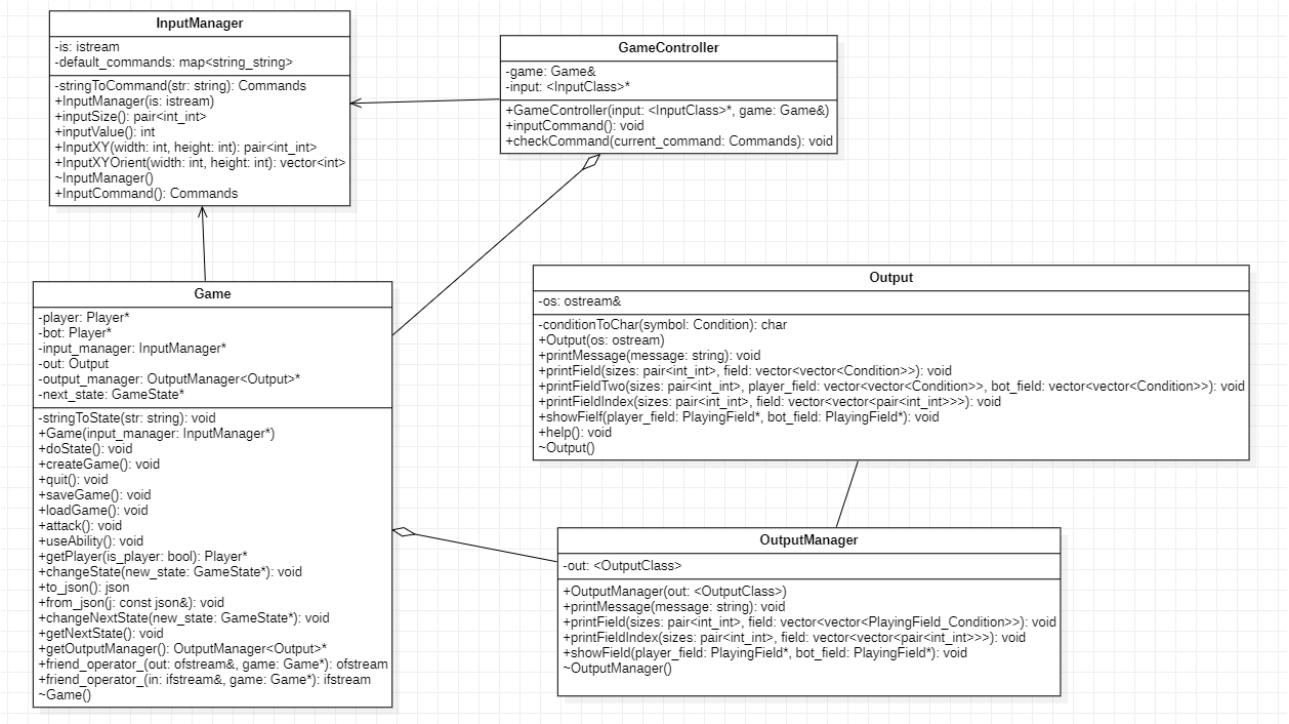

Рисунок 1. UML-диаграмма класса игры и новых классов

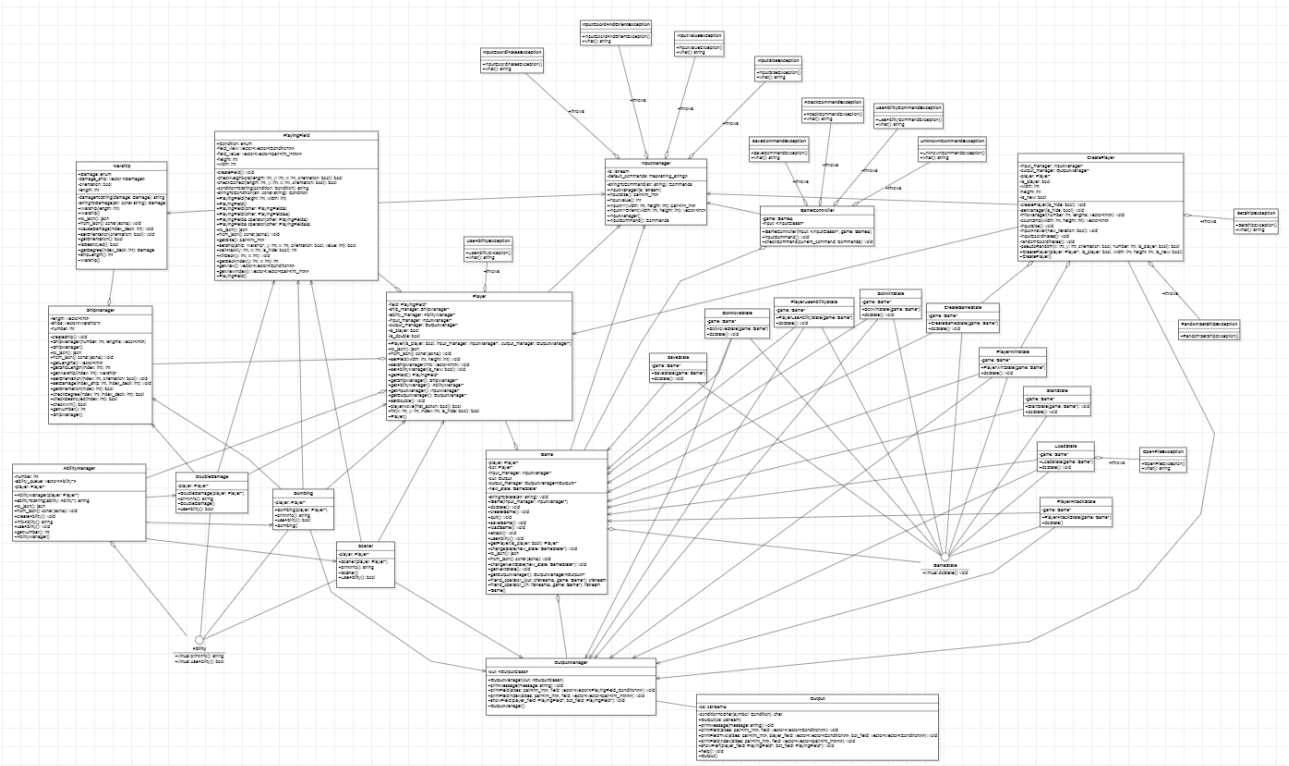

Рисунок 2. Общая UML-диаграмма классов

Класс *Output* производит вывод полей и сообщений в терминал. У него имеются такие методы, как *void printMessage(std::string message)* – для вывода сообщений, *void printField(..)* – для вывода поля одного игра. В него поступает поле, хранящее *enum Condition*, для преобразования их в символы,

выводящиеся в терминал существует метод *char conditionToChar(PlayingField::Condition symbol).* Кроме того есть метод *void printFieldTwo(...)* – для вывода полей двух игроков одновременно. Метод *void printFieldIndex(...)* – для вывода поля в формате чисел при расстановке кораблей. Метод *void showField(...),* который инициализирует вывод двух полей игроков. Также есть метод *void help(),* выводящий справку о командах.

Шаблонный класс *OutputManager* хранит в себе *out* - объект класса, поданного в качестве параметра шаблона *OutputClass*, который подается в конструкторе. Внутри есть методы: *void printMessage(...), void printField(...), void printFieldIndex(...), void showField(...)* и *void help(),* которые вызывают соответствующие методы класса, поданного в качестве параметра шаблона.

Для команд, вводимых пользователем создан файл *commands.json*, хранящий в себе соответствия команд действиям. Создан *enum Commands*, хранящий в себе передаваемые команды, такие как:

- *CREATE_GAME* – создать игру
- *SAVE_GAME* – сохранить игру
- *LOAD_GAME* – загрузить игру
- *NEXT_STATE* – выполнить следующее действие
- *QUIT* – завершить игру
- *ATTACK* – выполнить атаку
- *USE_ABILITY* – использовать способность
- *HELP* – вызвать справку

Класс *InputManager* создан для считывания ввода пользователя из терминала. В нем есть методы: *Commands inputCommand(),* который считывает команду из терминала, после чего ищет совпадение в файле с командами, если совпадение не найдено, вызывает ошибку и ждет ввода следующей команды. Если же совпадение найдено, то сопоставляет его с *enum* Commands при помощи метода *Commands stringToCommand(std::string str),* после чего возвращает команду, если невозможно открыть файл с командами, команды сопоставляются с командами из *std::map<std::string, std::string>*

default_commands.. Также имеются методы: *std::pair<int, int> inputSize()* – для считывание размеров поля при инициализации, *int inputValue()* – для считывания флага рандомной или ручной расстановки кораблей, *std::vector<int> inputXYOrient(int width, int height)* – для считывания координат и ориентации корабля при расстановке и *std::pair<int, int> inputXY(int width, int height)* – для считывания точки удара при атаке.

Шаблонный класс *GameController* хранит в себе ссылку на класс *Game* и указатель на класс ввода, поданного как параметр шаблона *InputClass*, все это передается в конструкторе. Метод *void inputCommand()* вызывает метод поданного класса ввода для получения команды в формате *enum Commands*, после чего передает полученную команду в метод *void checkCommand(Commands current_command),* в котором происходит проверка связи следующего состояния и поданной команды. Таким образом до создания игры нельзя вызвать методы сохранения, атаки, и использования способности, во время хода соперника нельзя использовать метод атаки или использования способности, после применения способности нельзя снова использовать способность. Выйти из игры, создать игру, вызвать следующее состояние, загрузить игру и вызвать справку можно в любой момент. Если команда не противоречит следующему состоянию, то вызывается соответствующий метод класса *Game*.

Выводы.

Были созданы: шаблонный класс управления игрой, шаблонный класс отображения игры, класс, считывающий ввод пользователя и класс, отвечающий за отрисовку поля.